

Data Structures and Fundamental Algorithms - #WEEK1

- 1) Order the following functions by growth rate: N , $\sqrt{N}$, $N^{1.5}$, N^2 , $N \log N$, $N \log \log N$, $N \log^2 N$, $N \log(N^2)$, $2/N$, 2^N , $2^{(N/2)}$, 37 , $N^2 \log N$, N^3 . Indicate which functions grow at the same rate.

Ans: ^[OBJ] The growing rate of functions given in question are in below ascending order

- 37 - It is the constant value, so it grows at slowest rate because it doesn't have to depend upon the value of "N"
- $2/N$ - As the value of N increases, the fraction value of this is decreases, so it grows very slow compared to any other value given
- $N \log \log N$ - This grows slower than the $\log N$ because the logarithmic value is again going to sub-logarithmised so the value is going to increase slowly.
- $\sqrt{N}$ - As this is a square root function, so it grows slower than linear function and faster than sub-log functions.
- N - This is a linear function so it's value will be dependent on the value of 'N'
- $N \log^2 N$ - As it is squared form of log so it's value less than $\log(N^2)$.
- $N \log(N^2)$ - As this is a multiply of logarithmic and linear function, it is faster than linear function. It simplifies to $2N \log N$, which is faster than linear but slower than quadratic.
- $N \log N$ - It is the multiplication of logarithmic value and linear value, so it is obviously greater than linear value.
- $N^{1.5}$ - It has the power as 1.5 for the linear function so its value is more than linear and increases exponentially with the increase in the value of N.
- N^2 - It is constantly increases with the increase in the value of 'N' - $O(N^2)$.
- $N^2 \log N$ - It is slightly greater than the value of N^2 but lesser than cubical growth
- N^3 - Cubical growth increases with increasing the value of N - $O(N^3)$.
- $2^{(N/2)}$ - This is exponential growth. But less than 2^N .
- 2^N - This is also exponential and faster than any other polynomial function.

The order of the growth rate as fallows,

Constant < $2/N$ < $N \log \log N$ < $\sqrt{N}$ < N < $N \log^2 N$ < $N \log(N^2)$ < $N \log N$ < $N^{1.5}$ < N^2 < $N^2 \log N$ < N^3 < $2^{(N/2)}$ < 2^N .

- 2) For each of the following six program fragments:
- Give an analysis of the running time (Big-Oh will do).
 - Implement the code in Java and give the running time for several values of N.
 - Compare your analysis with the actual running times.

```
(1) sum = 0;
    for( i = 0; i < n; i++ )
        for( j = 0; j < n; j++ )
            sum++;
```

Ans: a) Analysis of the running time: Number of computations are :-

Sum=0 – (1 Operation)

for(i = 0; i < n; i++) - i=0, i++ (Two operation), for loop – N operation

for(j = 0; j < n; j++) - for loop is N operation, j=0,j++ (Two operation)

Sum++ (One operation)

$f(N) = 1 + 2 + N(2+N) + 1$

$= 4 + 2N + N^2$

$= O(N^2)$.

```

1 public class FragOne{
2     public static void main(String[] args){
3         int n = 20;
4         int sum = 0;
5         long startTime = System.nanoTime();
6         for(i = 0; i < n; i++){
7             for(j = 0; j < i; j++){
8                 sum++;
9             }
10        }
11    }
12    }
13    }
14    }
15    }
16    }
17    }
18    }
19    }
20    }
21    }
22    }

```

Console Output:

```

<terminated> FragThree [Java Application] /Volumes/Eclipse/Eclipse.app/Contents/Eclipse/plugins/org.eclipse.justi.openjdk.hotspot.jre.full.macosx.aarch64_17.0.7.v20230425-1502/jre/bin/java (Sep 11, 2023, 6
For N = 10:
Sum : 55
Running Time: 0.007958 ms

```

Here, I have attached the running code of the given snippet 1

(2) $sum = 0;$ -- One operation

$for(i = 0; i < n; i++)$ -- $N + 2$

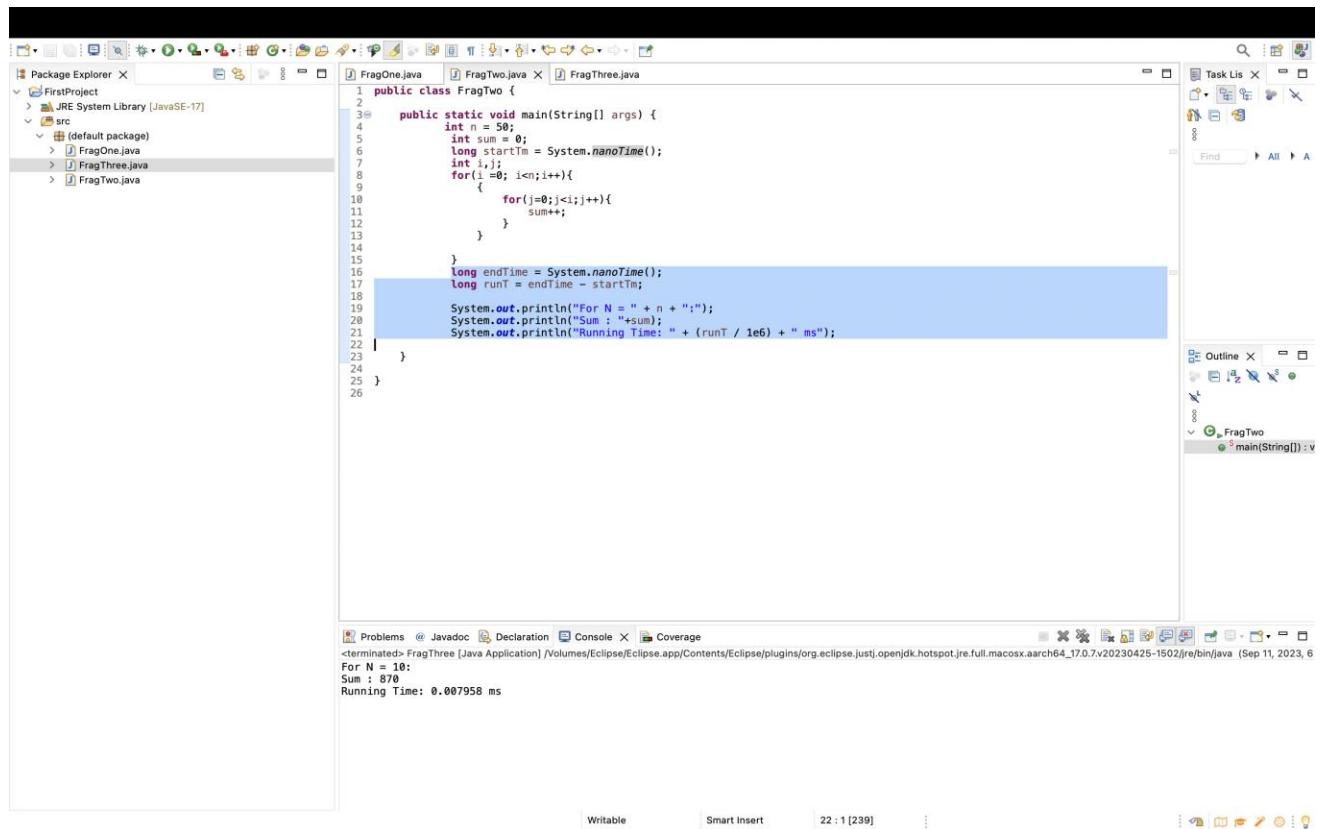
$for(j = 0; j < i; j++)$ -- $N + 2$

$sum++;$ -- 1

$$f(N) = 1 + 2 + N(N + 2) + 1$$

$$= 4 + N^2 + 2N$$

$$= O(N^2).$$



Here, I have attached the running code of the given snippet 2

(3) sum=0;	--	One Operation
for(i = 1; i < n; i++)	--	N + 2
for(j = 1; j < i * i; j++)	--	N + 2
if(j % i == 0)	--	One operation
for(k = 0; k < j; k++)	--	N + 2
sum++;	--	One operation

$$f(N) = 1 + 2 + N(N+2)(1)(N+2)+1$$

$$= O(N^3).$$

```

1 public class FragThree {
2     public static void main(String[] args){
3         int n = 10;
4         int sum = 0;
5         long startTm = System.nanoTime();
6         int i,j,k;
7         for( i = 1; i < n; i++){
8             for( j = 1; j < i * i; j++){
9                 if( j % i == 0 ){
10                     for( k = 0; k < j; k++){
11                         sum++;
12                     }
13                 }
14             }
15         }
16         long endTime = System.nanoTime();
17         long runT = endTime - startTm;
18         System.out.println("For N = " + n + ":");
19         System.out.println("Sum : "+sum);
20         System.out.println("Running Time: " + (runT / 1e6) + " ms");
21     }
22 }
23
24
25

```

Console Output:

```

<terminated> FragThree [Java Application] /Volumes/Eclipse/Eclipse.app/Contents/Eclipse/plugins/org.eclipse.justi.openjdk.hotspot.jre.full.macosx.aarch64_17.0.7.v20230425-1502/jre/bin/java (Sep 11, 2023, 6
For N = 10:
Sum : 870
Running Time: 0.007958 ms

```

Here, I have attached the running code of the given snippet 3.

3) An algorithm takes 0.5 ms for input size 100. How long will it take for input size 500 if the running time is the following (assume low-order terms are negligible):

- Linear
- $O(N \log N)$
- Quadratic
- Cubic

Ans:

A) If the Algorithm is linear then the running time will be increased linearly, so the linear running time of 500 is

For 100 - 0.5ms

Then for 500 - ?

So, $0.5 * 500 = 100 * ?$

Linear running time for 500 will be 2.5ms.

b) So, if the algorithm $O(N \log N)$ is compared then the running time will be higher than the linear.

Which is $0.5 / (100 * \log(100)) = \text{Runtime} / (500 * \log(500))$

Then, run time = 3.37ms which is greater than linear running time.

- c) Quadratic: So, for quadratic which is $O(N^2)$ which means run time will be increased by quadratic.

$$(0.5) / (100^2) = (\text{Run Time}) / (500^2)$$

So, the run time for the quadratic is going to be 12.5ms

- d) Cubic: (N^3) : Which is run time going to increase by cube.

$$(0.5) / (100^3) = (\text{Run Time}) / (500^3)$$

After solving it for Run time the value of cubic run time is 62.5ms.

- 4) Give an efficient algorithm to determine if there exists an integer i such that $A_i = 1$ in an array of integers $A_1 < A_2 < A_3 < \dots < A_N$. What is the running time of your algorithm.

Ans:

The given array is a sorted and this is the time where binary search comes into place to find out the value because by using this method, we can find out the value with a smaller number of iterations.

So, we are here to find the array value 1 in an ascending array of values from $A_1 < A_2 < A_3 < \dots < A_N$.

This algorithm involves in following steps to find out the values

First, in the sorted array it will take the middle value which is $(\text{low} + \text{high}) / 2$, so this indexed value going to be the median.

Then it checks if the key value which we needed matches with that or not, if yes then it will return the value, else it will check if the key value is higher than the mid or lower.

So, depending on the higher value or lower value the same process will continue. If the value is higher than it checks in the higher value series else, it will check with the lower value series and then same process will continue. Again, it will check for the middle value from the series and then compare it with that so on, this process will go on.

If the value is present in that then it will return that value else gives -1 as output.

The running value of the algorithm is $O(\log N)$, where N is the number of values in the array.

To find out maximum possibilities to find the value from the array of the values = $(\log_2 N + 1)$ is the formula, Where N is the number of values in the array.

Therefore, Binary search is an efficient algorithm for sorted data and faster than linear search.

